

ForgeGIS™

Technical Brief

VERSION 1.2.7 · OCTOBER 2026

Forged For Speed. Refined by Craft.

SEAGLASS FOUNDRY LLC

GPU-accelerated geospatial compute for Java and a native MCP server — measured on the WGS84 ellipsoid, checked against GDAL, and built so an agent's answer is the script's answer.

Executive Summary

ForgeGIS™ is a GPU-accelerated geospatial compute library for Java with a native Model Context Protocol (MCP) server. Its catalog covers terrain, bathymetry and ocean acoustics, hydrology, visibility and RF, spectral analysis and machine learning, interpolation, spatial geometry and topology, filtering and morphology, raster management and I/O, routing, point clouds and temporal analysis — callable directly from Java for scripted pipelines, and from AI agents through MCP. Raster data stays on the GPU between pipeline stages, and every operation has a CPU fallback for machines without a usable GPU. It ships as Maven artifacts with no GDAL and no native binaries of its own.

Release 1.2.6 is a release about being right. Every length, area, distance and bearing now comes from one geodesic engine on the WGS84 ellipsoid. Outputs that have a GDAL counterpart are checked against GDAL, and the results are stated with their scope. Every raster operation is held to a CPU/GPU agreement contract, a NoData sweep and a US-survey-foot check. Bad input is refused up front, with a typed reason an agent can branch on, and an operation that will not fit in memory is refused before anything is allocated.

15×	**6×**	**0**	**WGS84**
geometric-mean speedup over GDAL 3.12.1 across 242 operations at sizes from 16×16 to 8192×8192 pixels — 239 faster, 3 slower, all tiny†	kernel against kernel: GDAL in process, operations where both take 1 ms or more, 256×256 to 4096×4096 plus real elevation tiles†	parity failures: 1,191 checks pass, 0 fail; 884 compare the output with GDAL's or an independent oracle†	for every measurement, by Karney's method in one geodesic engine, and each result names its model

† The 1.2.6 benchmark of 2026-10-04 (forgegis-validation run 20261004-094534-191f1b44): ForgeGIS 1.2.6 on a laptop RTX 5070, Windows 11, warm timings, GDAL 3.12.1. Every speed and parity figure in this brief is from that run; see Performance.

What is new since the June brief

- **1.2.0** — `run_dag`: branching pipelines across raster, vector and point-cloud stages in one MCP call; zonal statistics as a raster ↔ vector bridge; select-by-location; stream topology including height above nearest drainage; tasseled cap for Sentinel-2; bounded and memory-mapped I/O; cooperative cancellation. The licence was finalized as proprietary.
- **1.2.3** — the digital twin: record, replay, fork and what-if over real terrain, with certified physics operations, a wire serving path and shared-VRAM co-residency; op-execution provenance on every result; tiled execution byte-identical to untiled.
- **1.2.4** — a correctness and contract release: a source with no usable coordinate system is refused rather than guessed; a real Cloud-Optimized GeoTIFF writer; client cancellation.
- **1.2.5** — `sample_point` (one cell, all bands or a small window around it, without reading the whole raster); the twin's feature-delta lane; MCP results bounded before they reach a language model.

- **1.2.6** — the accuracy release described in this brief.

What ForgeGIS Is

ForgeGIS is a compute substrate for two audiences whose needs are converging: teams that run professional GIS pipelines today, and teams that want AI agents to run geospatial analysis tomorrow. It does not replace the analyst's desktop GIS. It replaces the layer underneath — the engine that runs the operation the analyst, the script or the agent asks for — and it gives the script and the agent the same engine.

Within the Forge suite, ForgeGIS is the compute engine: the Seaglass Control Center™ installs it, ForgeMind™ runs it as an MCP child process, ForgeData™ supplies its data, and ForgeGIS Studio™ and Seaglass Globe™ show its results. Outside the suite it is a set of Maven artifacts that any JVM service can depend on.

Architecture

GPU-resident pipelines

Raster data lands on the GPU once and stays there across a pipeline, moving between stages as device buffers rather than round-tripping through host memory or disk. Inter-stage I/O is the dominant cost of a pipeline built from command-line tools; removing it is the structural source of the pipeline speedups in the Performance section. Pipelines can be linear chains or directed acyclic graphs: raster, vector and mixed executors run graphs in which one stage feeds several consumers and several inputs converge on one operation, and since 1.2.0 `run_dag` carries point-cloud stages too. The SavedPipeline format holds typed slot references, conditionals and nested pipelines; it is the same document ForgeGIS Studio saves.

QGIS model import

A dedicated module reads QGIS `.model3` processing models and compiles them to ForgeGIS graphs, with an `--inspect` report that names any unsupported algorithm up front. Expressions run through a clean-room QGIS Processing expression interpreter — its own lexer, parser and evaluator over a whitelisted function set.

Distribution without native installs

ForgeGIS is Java: a set of Maven modules under `com.seaglassfoundry` plus a bill of materials, built in one direction — the compute core depends on neither the I/O layer nor the MCP server — so a consumer can take the raster library without the server, or the server as one shaded jar. There is no GDAL at runtime and no native install step. GPU dispatch goes through the JOCL OpenCL bindings, whose JNI bridge ships inside the JOCL jar; the system OpenCL runtime comes with the GPU driver. ForgeGIS ships no native binaries of its own. The optional twin co-residency module uses Java's foreign-function API, and only when it is enabled.

One engine, two surfaces

Every operation is a Java method. The MCP server is a parameter-marshalling layer over the same implementations, not a second implementation, so an agent's result depends on the operation and its parameters, not on who called it. K-means and random forest default to a fixed seed, so agent runs repeat without per-call seed configuration; the Java API exposes the seed. Tiled execution is byte-identical to untiled.

The Agent Surface

The MCP server speaks JSON-RPC over stdio. It reaches the catalog two ways: a **pipeline surface** — `list_ops` to browse the composable ops, and `run_pipeline`, `run_vector_pipeline` and `run_dag` to execute a chain or graph of them in one call, with intermediates staying on the GPU — and **typed intent tools** for common single-call tasks such as viewshed, hydrology, interpolation, point-cloud processing, routing and geodesic measurement. A handful of primitives load, describe, write and release rasters by handle.

Profile	What `tools/list` shows	Chosen by
`minimal` *(default)*	the primitives and the pipeline tools, so an agent's tool list starts small	default
`standard`	the primitives, the intent tools and the pipeline tools	`--profile standard` or `FORGEGIS_MCP_PROFILE`

The profile governs discovery, not capability: a tool missing from `tools/list` is still callable by name. A few further tools — the digital-twin verbs, the op-surface binding verbs and `contract_info` — are never listed under either profile and are callable by name.

A versioned contract. The MCP surface is recorded in a generated contract snapshot: every tool's input schema and applied defaults, every composable op's parameters, the output keys, each op's NoData rule, and the closed error vocabulary. A build gate fails on any change that is not reflected in a version bump, and `contract_info` reports the version to a running client.

Errors an agent can branch on. A failure carries an `error.category` — `TOOL_ERROR`, `SANDBOX_DENIED`, `TOOL_TIMEOUT`, `SERVER_BUSY` or `CANCELLED` — and, where it matters, an `error.reason`: `OUT_OF_MEMORY`, `INSUFFICIENT_GPU_MEMORY`, `INPUT_TOO_LARGE`, `INSUFFICIENT_HEAP`, `POINT_OUTSIDE`, `SOURCE_OUTSIDE`, `SOURCE_NODATA`, `DESTINATION_OUTSIDE`, plus the sandbox reasons. Arguments are type- and range-checked, and no error message names a Java internal.

Refused before it runs. Every pipeline, graph and single-op entry compares the operation's measured heap demand with a budget taken from the free heap and refuses with `INSUFFICIENT_HEAP` when it would not fit — the op never runs and nothing is allocated for it. The estimates are measured, and a test gates their accuracy across the registry.

Bounded by design. Reads and writes are confined to configured roots (`FORGEGIS_READ_ROOTS`, `FORGEGIS_WRITE_ROOTS`); every tool call has a timeout; results are bounded before they reach a language model.

What Changed in 1.2.6

Every measurement on WGS84

Lengths, areas, perimeters, distances and bearings now come from one geodesic core built on GeographicLib, using Karney's method on the WGS84 ellipsoid, in place of the spheres used before. Polygon areas are correct at the poles and across the antimeridian. Metres mean metres in any coordinate system: buffers, offset lines, split and relate, profiles, distance rasters, the viewshed radius, line of sight, Fresnel clearance, stream length and pour-point snapping all measure on the ellipsoid, and cell areas and volumes use WGS84 ground area. Edges can be geodesic (the default), rhumb, linear or planar, and every result names the model it used.

Checked against GDAL — with the scope stated

Outputs with a GDAL counterpart are compared with GDAL, and the other operation classes with the reference that fits them (see Validation). In the 1.2.6 benchmark of 2026-10-04 (run 20261004-094534-191f1b44), **1,191 checks passed and 0 failed**: 884 compare ForgeGIS's output with GDAL's or an independent oracle, within a tolerance stated per operation; 294 are consistency checks that do not count as parity evidence, and 13 are smoke checks only. Every output was also re-run and compared for determinism.

Viewshed is held to agreement, not identity: ForgeGIS samples each sight line against a bilinear surface and GDAL sweeps line by line, and the cells where they differ are that documented difference. The evidence of record for viewshed is an exact ray trace of every sight line, which is gated: ForgeGIS never reports a cell hidden where the surface shows it. Viewshed also no longer steps over thin ridges, and the observer sits exactly where it was given.

Held to contract across the catalog

- **CPU and GPU agree.** Every raster op runs on both the CPU and the OpenCL backend over a shared set of inputs, each held to its declared contract — exact by default, a stated ULP tolerance where floating-point order differs. A new op cannot skip the test.
- **NoData is handled the same way everywhere.** Every raster op runs with voids encoded in each of the common ways, as untagged data, and as a multi-band input with per-band NoData, on both backends. Each op must declare its NoData rule, voids must stay voids, and an integer output always carries a NoData value it can store. The known defects the sweep still finds are pinned in the test and tracked.
- **US-survey-foot DEMs.** A projected DEM's heights are read in its coordinate system's unit. Every raster op is run on a metre grid and a US-foot grid of the same terrain and must agree. The defects it found — a buffer in metres that covered feet, volumes too large by the foot-to-metre factor, a viewshed mast measured in feet — are fixed.

- **Fuzzed.** GeoTIFF, LAS/LAZ and shapefile readers were hardened against hostile files, and the MCP surface against fuzzed arguments: oversized counts no longer hang the server or exhaust its heap.

Corrected algorithms

Operations now run the algorithm their name promises: natural-neighbour interpolation is true Sibson (it was a nearest-neighbour blend); the spline is a real thin-plate spline with an affine term; TIN interpolation builds a real Delaunay triangulation and leaves cells outside the hull as NoData; and the point-cloud ground filter defaults to a progressive morphological filter with PDAL's parameters (the old rule stays available). Fill NoData defaults to GDAL's inverse-distance strategy, rebuilt white-room from black-box probes, on the GPU as well as the CPU.

Changes an integrator must act on

- **Renamed tool.** `compute_haversine_distance` is now `compute_geodesic_distance`. No alias is kept: the old name gets an unknown-tool error, and the default answer moves from the sphere to WGS84 (`metric: HAVERSINE` still gives the sphere).
- **Viewshed output is UINT8.** Visible is 255 and not visible 0, as `gdal_viewshed` writes it; when the DEM has voids they are tagged NoData, 254 by default. `outputDataType: "float32"` restores the earlier 1.0 / 0.0.
- **Point parameters default to EPSG:4326.** Every point parameter on the MCP surface is read in its `crs`, which defaults to longitude/latitude. A point off the raster is refused (`POINT_OUTSIDE`, `SOURCE_OUTSIDE`, `DESTINATION_OUTSIDE`) rather than clamped. Java calls without a CRS keep the raster's own.

Capability Catalog

The catalog is grouped as below; operations are named, not counted.

Terrain. Slope (Horn, Zevenbergen–Thorne, Williams), aspect, hillshade, curvature, TRI, TPI, VRM, roughness, rugosity, surface-area ratio, heat load, convergence, landform classification, and vertical and horizontal datum conversion — EGM96 ships with the library.

Bathymetry and ocean. Bathymetric position index, benthic terrain and backscatter classification, benthic irradiance, depth uncertainty, isopach, dredge volume, wave exposure, reef complexity, and ocean acoustics: sound-velocity profiles, the SOFAR channel, surface ducts.

Hydrology. Sink fill, D8 flow direction, flow accumulation, stream network and Strahler order, watershed delineation, topographic wetness, and stream topology with height above nearest drainage.

Visibility and RF. Viewshed, batched multi-observer viewshed, intervisibility, Fresnel-zone clearance, line-of-sight profile, horizon angle, insolation with terrain shadowing, and shadow.

Spectral and machine learning. Band math and the common indices, tasseled cap, PCA, MNF, pansharpening, spectral angle mapping, unmixing, GLCM texture, K-means, maximum likelihood, random forest, DBSCAN and HDBSCAN.

Spatial geometry. The largest category: buffering, overlay, dissolve, Voronoi, hulls, simplification, field calculators, proximity and k-nearest-neighbour search, spatial joins, trajectory analysis, and the full DE-9IM surface — named predicates, mask-driven Relate and the raw intersection matrix for every geometry-type pair.

Filtering and morphology. Focal statistics, Gaussian, median and majority filters, edge detection, convolution, morphology, sieve, nibble, spike removal and NoData filling.

Raster management and I/O. Map algebra, reclassify, resample, reproject and a full `gdalwarp` equivalent, translate, clip and cutline, mosaic, rasterize, polygonize, contours, zonal statistics, overviews; GeoTIFF and COG (including HTTP and cloud range reads), NetCDF, GeoPackage, BAG, SRTM, GeoJSON, Shapefile, KML and CSV.

Interpolation, routing, point cloud and temporal. IDW, kriging with variogram fitting, Sibson natural neighbour, thin-plate spline and TIN; travelling salesman, multi-depot and capacitated and time-windowed vehicle routing; LAS/LAZ, COPC and EPT reads, ground filtering, classification, canopy height and voxelization; temporal statistics and anomaly detection.

The complete enumerated catalog, with each operation's GDAL equivalent and API surface, is available on request.

Validation

ForgeGIS is validated by a separate harness, against references chosen per operation class rather than one universal oracle: **GDAL** for raster operations with a direct equivalent; **JTS** for planar predicates and topology; **GeographicLib** and PROJ's `geod` for geodesy; **PDAL** for point clouds; **scipy, numpy, scikit-learn and PyKrige** for numerical and statistical operations; an **exact visibility ray trace** for line of sight; and **closed-form fixtures** where no external reference exists. Where an engine and a reference differ by design, the difference is written down as a known divergence rather than tuned away.

Determinism is checked by re-running each benchmarked operation and comparing outputs: in the 1.2.6 benchmark of 2026-10-04 (run 20261004-094534-191f1b44) it passed 1,191 of 1,191 records. Every cookbook recipe in the user guide is compiled and run by the build.

Performance

The figures in this section are **the 1.2.6 benchmark of 2026-10-04**, the canonical run `20261004-094534-191f1b44` of the forgegis-validation harness — a clean, tagged build on the reference laptop below, GDAL 3.12.1. All timings are warm medians: each operation's warm iterations, cold first calls excluded, with the GPU kept in its working state

between timed runs. **The benchmark's timing method was tightened in this release, so earlier published ForgeGIS figures are not directly comparable and should not be read as a trend.**

Single operations against GDAL

Across every operation with a GDAL equivalent where both sides were timed — **242 benchmark operations, no category excluded**, at sizes from 16×16 to 8192×8192 pixels — ForgeGIS's geometric-mean speedup over GDAL 3.12 was **15×** (14.97×), with **239 faster and 3 slower**; a geometric mean, not an average. The 3 slower were all tiny: raster creation and rasterize at 256×256, and raster creation at 1024×1024. The median operation was 18× faster. By raster size it was 11.6× at 256×256, 9.2× at 1024×1024 and 14.8× at 4096×4096.

The defensible kernel figure sits beside it: against GDAL running in process, on operations where both sides take at least a millisecond (256×256 to 4096×4096 plus real elevation tiles), ForgeGIS was **6× faster** (6.23×) — kernel against kernel, with no timer-resolution advantage to either side. Against the GDAL command line, including process start, the 30 shell-out operations read 178× — a workflow cost, not a kernel cost, and not comparable with the figures above.

Task	Operations	Geometric mean
Pipelines	14	81×
Resampling	51	30×
Transforms	13	30×
Vector	44	26×
Warps	20	24×
Zonal / focal	29	20×
Terrain	5	4.3×
Viewshed	2	3.4×
File I/O	64	3.0×

The 1.2.6 benchmark of 2026-10-04, warm medians; operations are operation-and-raster-size measurements, and each task's geometric mean spans every size it was run at, small rasters included.

Multi-stage pipelines

The pipeline benchmarks compare ForgeGIS executing a chain on the GPU with the same chain run as GDAL command-line tools writing a temporary GeoTIFF between stages — the usual way such workflows are orchestrated outside an in-process library. That makes this a workflow comparison, not a kernel comparison.

Pipeline chain	Raster	GDAL CLI chain	ForgeGIS	Speedup
Algebra — band math → focal → reclassify	1024²	214 ms	2.46 ms	87×
Algebra	4096²	709 ms	14.9 ms	47.5×
Warp — reproject → resample → clip → scale	4096²	1,348 ms	7.42 ms	182×
Terrain — slope → aspect → hillshade → reproject	1024²	332 ms	8.82 ms	37.6×
Terrain	4096²	1,624 ms	146 ms	11.1×
Graph — linear slope chain	1024²	80.7 ms	4.41 ms	18.3×
Graph — linear slope chain	4096²	348 ms	29.2 ms	11.9×
Graph — diamond: slope and aspect → band math‡	1024²	175 ms	1.94 ms	90.4×
Graph — diamond‡	4096²	813 ms	12.0 ms	67.9×
Graph — variadic band concatenation	1024²	157 ms	8.93 ms	17.5×

The 1.2.6 benchmark of 2026-10-04, warm medians; rows where ForgeGIS took under a millisecond are left out. At 4096² the pipelines ran 11–182× ahead of the equivalent GDAL CLI chains. ‡ The two diamond rows sit outside the 242-operation headline cohort because their reference, GDAL+Sum, is not plain GDAL: it is two `gdaldem` command-line runs (slope and aspect) followed by a cell-wise sum of their outputs that the benchmark harness computes in process.

What is not claimed

Small single operations are not where ForgeGIS's case rests. At 1024×1024 and below, a GPU dispatch has little work to amortize: the run's 3 slower operations were all tiny, and many other small rows are only narrowly ahead; sub-millisecond operations can also be slower than GDAL on the first call after the GPU has gone idle, so these timings are not a claim about interactive latency. ForgeGIS's performance claims are about multi-stage pipelines and large rasters. Format-conversion round-trips are dominated by I/O and are not counted as compute wins. In the pipeline rows GDAL runs as command-line tools, so its process start is part of its time — a fair cost of that workflow, and why those rows are a workflow claim; in the other single-operation records GDAL runs in process, kernel against kernel. Heap and memory figures are not published for this release.

Reference hardware

One laptop, as recorded by the run: Intel Core 7 240H, NVIDIA GeForce RTX 5070 Laptop GPU (driver 573.22), Windows 11, OpenJDK Corretto 26 with a 4 GB heap, GDAL 3.12.1. ForgeGIS needs no server-class hardware to produce these results.

Deployment

Runtime. Java 17 or later (the build uses JDK 25). A GPU with an OpenCL runtime — the device picker prefers a discrete NVIDIA, then AMD, then Intel GPU — or none: every operation has a CPU fallback, and a GPU without double precision routes the operations that need it to the CPU. No GDAL installation. Validated on Windows 11 with an NVIDIA RTX 5070 Laptop GPU; other configurations are not validated for this release.

The MCP server is one shaded jar launched as a process: `java --enable-native-access=ALL-UNNAMED -jar forgegis-mcp-<version>-all.jar [--profile standard]`. The native-access flag silences the JDK's warning about JOCL's library load on recent JDKs; warnings go to stderr, so the JSON-RPC stream on stdout is unaffected. In the Forge suite the Seaglass Control Center installs the engine with a private Java runtime and ForgeMind starts it.

Restricted environments. With no GDAL, no system-level native install and no native binaries of its own, ForgeGIS reaches an air-gapped or accredited host as jars. What the host must already provide — a JDK and a GPU driver with its OpenCL runtime — has established approval paths. File access is confined to the configured read and write roots.

Specifications at a glance

Version	ForgeGIS 1.2.7 (October 2026); the benchmarks in this brief were run on 1.2.6
Catalog	terrain, bathymetry and ocean, hydrology, visibility and RF, spectral and ML, spatial geometry, filtering, raster management and I/O, interpolation, routing, point cloud, temporal; composable in pipelines and DAGs
MCP	stdio JSON-RPC; default `minimal` and full `standard` profiles; versioned contract snapshot; typed `error.category` and `error.reason`
Geodesy	WGS84 by Karney's method (GeographicLib) for every measurement; geodesic, rhumb, linear or planar edges
Compute	OpenCL through JOCL; GPU-resident pipelines and DAGs; CPU fallback for every op
Formats	GeoTIFF, COG (local, HTTP, S3, GCS, Azure range reads), NetCDF, GeoPackage, BAG, SRTM, PNG/JPEG + world file, GeoJSON, Shapefile, KML/KMZ, CSV, WKT/WKB/MVT, LAS/LAZ, COPC, EPT; QGIS `.model3` import
Benchmark	the 1.2.6 benchmark of 2026-10-04, run 20261004-094534-191f1b44: 15× geometric mean over GDAL 3.12.1 across 242 operations (239 faster, 3 slower, all tiny); 6× kernel against kernel
Platform	Java 17+; validated on Windows 11 with an NVIDIA RTX 5070 Laptop GPU
Distribution	Maven artifacts, `com.seaglassfoundry`, modules and a bill of materials; one shaded MCP jar
Licence	Proprietary, by written agreement; evaluation builds available

Licensing and Contact

ForgeGIS is proprietary software of Seaglass Foundry LLC: it is licensed, not sold, and no rights are granted except under a separate written agreement. Evaluation builds are available to qualified teams, and nothing in this brief is a grant of rights. Third-party components keep their own licences, listed in the NOTICE file that ships with the jars. GeoTools 29.1 (LGPL 2.1), whose replacement by ForgeGIS's own readers is planned, and laszip4j 0.21 (LGPL 3.0 or later) are bundled unmodified and unrelocated into the shaded MCP jar, where they can be replaced. The EGM96 geoid is a bundled public-domain dataset.

Contact. rich@seaglassfoundry.com · seaglassfoundry.com

ForgeGIS™, ForgeMind™, ForgeData™, ForgeGIS Studio™, Seaglass Globe™, Seaglass Control Center™, and Seaglass Foundry™ are trademarks of Seaglass Foundry LLC. GDAL, PDAL, JTS, GeoTools, GeographicLib, PROJ, QGIS, OpenCL, JOCL, NVIDIA, GeForce, RTX, Intel, Microsoft, Windows, Java, OpenJDK, Corretto, Apache Maven, Copernicus, and the Model Context Protocol are trademarks or marks of their respective owners. Reference to these marks does not imply endorsement. © 2026 Seaglass Foundry LLC. All rights reserved.